FICHA DE IDENTIFICACIÓN DE TRABAJO DE INVESTIGACIÓN

Título	Interpretabilidad en Redes Neuronales Mediante Analogías con Redes Biológicas	
Autor/es	**Nombres y Apellidos**	**Código de estudiantes**
	Emilio Gabriel Fernández Jiménez	88078
Fecha	21/04/2026	

Carrera	Ing. De Sistemas
Asignatura	Redes Neuronales
Grupo	A
Docente	Ing. Roger Rolando Perez Rojas
Periodo Académico	I/2026
Subsede	Oruro

Copyright © (2026) por (Emilio Gabriel Fernández Jiménez). Todos los derechos reservados.

Tabla De Contenidos

Lista De Tablas 3
Lista De Figuras 4
Introducción 5
Capítulo 1. Planteamiento del Problema 6
1.1 Formulación del Problema 6
1.2 Objetivos 6
1.3 Justificación 6
1.4 Planteamiento de Hipótesis 7
Capítulo 2. Marco Teórico 9
2.1 Área de Estudio/Campo de Investigación 9
2.2 Desarrollo del Marco Teórico 9
2.2.1 Arquitectura y Transmisión de Señales en BNN 9
2.2.2 Modelado Matemático en Redes Neuronales Continuas (ANN) 11
2.2.3 Opacidad por Profundidad (Deep Learning) 13
2.2.4 Riesgos de la Opacidad en el Software Crítico 15
2.2.5 Visualización de Mapas de Características (Feature Maps) 17
2.2.6 Aislamiento de Estímulos y Análisis de Sensibilidad 20
2.2.7 Explicabilidad Global vs. Local 22
2.2.8 Atribución de Características mediante Teoría de Juegos (SHAP) 24
Capítulo 3. Resultados y Discusión 28
3.1 Funciones de Soporte Vitales para el Método Científico 28
3.2 Codificación 29
3.3 Aplicación 49
Capítulo 4. Conclusiones 50
Referencias 52

Lista De Tablas

Tabla 1. Comparativa Estructural entre Redes Biológicas y Artificiales 11
Tabla 2. Funciones de Activación y su Impacto en la Explicabilidad 13
Tabla 3. Relación entre Complejidad de Arquitectura y Opacidad 15
Tabla 4. Análisis de Riesgos por Sector Industrial 17
Tabla 5. Analogía de Técnicas de Observación Interna 19
Tabla 6. Comparativa de Intervención por Perturbación 22
Tabla 7. Diferencias Arquitectónicas: Explicabilidad Global vs Local 24
Tabla 8. Interpretación de los Valores SHAP en la Ingeniería 26

Lista De Figuras

Figura 1. Arquitectura y Transmisión de Señales en BNN 10
Figura 2. Modelado Matemático en Redes Neuronales Continuas (ANN) 12
Figura 3. Opacidad por Profundidad (Deep Learning) 14
Figura 4. Riesgos de la Opacidad en el Software Crítico 16
Figura 5. Visualización de Mapas de Características (Feature Maps) 18
Figura 6. Aislamiento de Estímulos y Análisis de Sensibilidad 21
Figura 7. Explicabilidad Global vs. Local 23
Figura 8. Atribución de Características mediante Teoría de Juegos (SHAP) 25
Figura 9. Informe 49
Figura 10. Gráfico 49

Introducción

Las redes neuronales artificiales continuas, tales como los perceptrones multicapa (MLP) o las redes convolucionales (CNN), han demostrado una capacidad excepcional para aproximar funciones no lineales y resolver problemas complejos de clasificación y regresión.

Sin embargo, a medida que la arquitectura de estos modelos se vuelve más profunda y parametrizada, su proceso interno de toma de decisiones se vuelve matemáticamente opaco.

Este fenómeno, conocido en el ámbito del desarrollo de software como el problema de la "caja negra", representa un obstáculo crítico cuando se busca implementar Inteligencia Artificial en entornos empresariales, médicos o industriales donde la trazabilidad y la auditoría son obligatorias.

El presente trabajo de investigación aborda el Tema 7 del laboratorio, enfocado en la interpretabilidad de redes neuronales continuas mediante analogías con redes biológicas.

A través del campo de la Inteligencia Artificial Explicable (XAI), se explora cómo es posible mapear e interpretar las representaciones internas de una red artificial emulando las metodologías que emplean los neurocientíficos para rastrear estímulos en el cerebro humano.

El objetivo es demostrar que la apertura de esta "caja negra" no solo es teóricamente posible, sino que es un requisito de ingeniería indispensable para garantizar la confiabilidad del software.

Capítulo 1. Planteamiento del Problema

1.1 Formulación del Problema

¿De qué manera la implementación de técnicas de Inteligencia Artificial Explicable (XAI), basadas en analogías con el procesamiento de estímulos en redes neuronales biológicas, permite mitigar el problema de la "caja negra" en redes neuronales artificiales continuas y aumentar la confiabilidad en sistemas de software comercial e industrial?

1.2 Objetivos

Analizar y proponer un marco de interpretabilidad para redes neuronales artificiales continuas mediante técnicas de XAI, estableciendo paralelismos teóricos y algorítmicos con las redes neuronales biológicas para la atribución de características en software predictivo.

- Comparar la dinámica de activación continua en las Redes Neuronales Artificiales (ANN) frente al procesamiento de estímulos en las Redes Neuronales Biológicas (BNN).
- Evaluar métodos de explicabilidad local (como SHAP o LIME) que simulan el aislamiento de rutas neuronales para determinar qué variables de entrada detonan una predicción específica.
- Diseñar una propuesta de implementación en lenguaje Python que evidencie la apertura de la “caja negra” en un modelo de red neuronal continua.

1.3 Justificación

Desde la perspectiva de la Ingeniería de Sistemas, el éxito de un modelo predictivo no se limita a obtener una alta precisión métrica (accuracy) durante su entrenamiento. En escenarios de despliegue real —como la predicción de demanda e inventario en sistemas comerciales bajo restricciones (véase el Caso Delizia)—, los tomadores de decisiones necesitan comprender por

qué el algoritmo recomienda una acción específica. Depender de un modelo inescrutable para invertir capital o gestionar recursos es un riesgo inaceptable.

Aquí es donde la analogía biológica adquiere un valor práctico invaluable. De la misma manera en que la neurociencia utiliza herramientas de escaneo para observar cómo se ilumina una región del cerebro ante un estímulo visual o auditivo, la ingeniería de software utiliza técnicas de XAI para rastrear qué variables de entrada "iluminan" o activan los nodos clave de una red neuronal artificial.

Lograr esta explicabilidad permite depurar (debuggear) los modelos, identificar sesgos ocultos, cumplir con normativas de transparencia y, en última instancia, construir software en el que los usuarios finales puedan confiar plenamente.

1.4 Planteamiento de Hipótesis

La presente investigación parte de la premisa de que la opacidad en las redes neuronales artificiales profundas es una limitación técnica superable mediante la instrumentación de auditoría adecuada. Bajo este marco, se plantea la hipótesis de que la implementación de técnicas de Inteligencia Artificial Explicable (XAI), fundamentadas en la analogía neurobiológica de mapeo de estímulos y respuestas, permite descomponer el comportamiento interno de los modelos continuos en explicaciones locales comprensibles.

Este enfoque no solo mitiga el problema de la "caja negra", sino que garantiza una validación técnica rigurosa indispensable para el despliegue de sistemas de software en entornos de misión crítica.

Operativamente, se postula que el uso de frameworks como SHAP o LIME, inspirados en los principios de asignación de crédito y observación funcional de la neurociencia, permite transformar tensores abstractos en indicadores de causalidad alineados con la lógica de negocio.

Esta capacidad de interpretación facilita la identificación de sesgos y la corrección de errores antes de la producción, asegurando que las decisiones algorítmicas —como las aplicadas en la optimización de inventarios del Caso Delizia— sean transparentes y verificables.

En última instancia, la hipótesis sostiene que la explicabilidad es el componente esencial que eleva la precisión métrica al nivel de una solución de ingeniería robusta, confiable y comercialmente viable.

Capítulo 2. Marco Teórico

2.1 Área de Estudio/Campo de Investigación

Área de estudio: Ingeniería de Sistemas – Redes Neuronales

2.2 Desarrollo del Marco Teórico

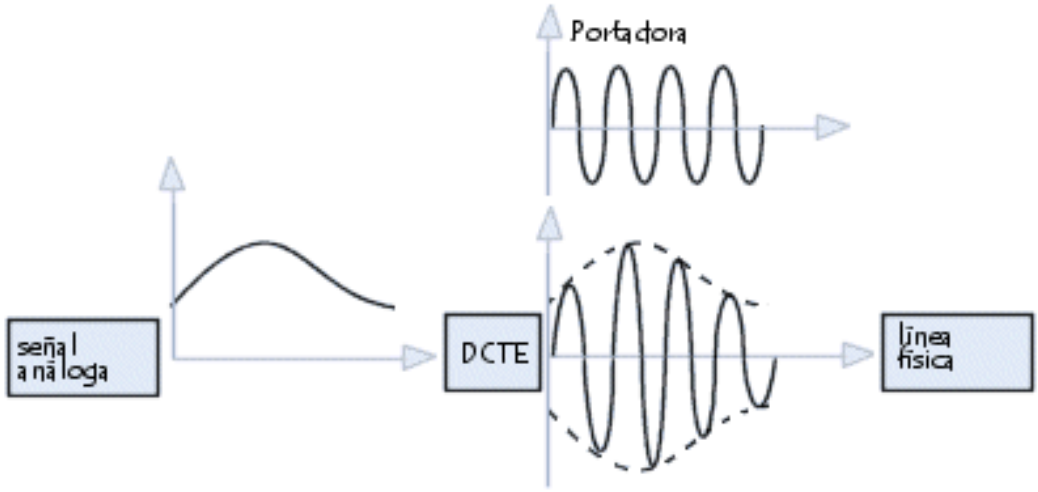
2.2.1 Arquitectura y Transmisión de Señales en BNN

La arquitectura del cerebro humano es el sistema de procesamiento de información más complejo conocido, compuesto por aproximadamente 86,000 millones de neuronas. Estas unidades biológicas no operan de forma aislada, sino que forman redes densamente interconectadas a través de sinapsis.

La transmisión de señales en estas redes es de naturaleza electroquímica; cuando una neurona recibe suficientes estímulos en sus dendritas, genera un potencial de acción que viaja por el axón para liberar neurotransmisores, afectando a la siguiente neurona en la cadena.

A diferencia de los sistemas digitales, las redes biológicas (BNN) son intrínsecamente ruidosas y estocásticas. Esto significa que una misma entrada no siempre produce exactamente la misma salida, debido a factores como el cansancio sináptico o la presencia de diferentes niveles hormonales. Sin embargo, esta aparente ineficiencia es lo que otorga al cerebro su capacidad de generalización y resistencia a fallos.

Figura 1. Arquitectura y Transmisión de Señales en BNN



Nota. Imagen obtenida de repositorios digitales de libre distribución (Autor desconocido).

Si una pequeña fracción de neuronas muere, el sistema redistribuye la carga de trabajo, una propiedad conocida como plasticidad neuronal.

Desde el punto de vista de la interpretabilidad, las BNN son "legibles" mediante técnicas de imagenología médica. Por ejemplo, se sabe que el procesamiento visual ocurre en la corteza occipital de manera jerárquica: unas neuronas responden a líneas, otras a formas y finalmente otras a rostros.

Este orden biológico es el que intentamos replicar en el software para entender qué "ve" el algoritmo cuando analiza datos de entrada.

Finalmente, es vital comprender que el aprendizaje en las BNN ocurre mediante la potenciación a largo plazo (LTP), donde las conexiones sinápticas se fortalecen físicamente con el uso. Este proceso es continuo y asíncrono, lo que permite al ser humano aprender en tiempo real a partir de un solo ejemplo, a diferencia de los modelos artificiales que requieren miles de iteraciones sobre los mismos datos.

Tabla 1. Comparativa Estructural entre Redes Biológicas y Artificiales

Característica	**Redes Neuronales Biológicas (BNN)**	**Redes Neuronales Artificiales (ANN)**
Unidad de Proceso	Neurona biológica (viva)	Nodo matemático (función)
Señal de Transmisión	Impulsos electroquímicos (Spikes)	Valores numéricos continuos (Floats)
Aprendizaje	Sináptico (Potenciación química)	Ajuste de pesos (Backpropagation)
Consumo Energético	Muy bajo (~20 Vatios)	Muy alto (Requiere GPUs potentes)
Procesamiento	Masivamente paralelo y asíncrono	Secuencial/Paralelo sincrónico

Nota. Elaboración propia. Comparación estructural fundamentada en las diferencias físicas y lógicas descritas por la neurociencia y los modelos de Deep Learning (Goodfellow et al., 2016).

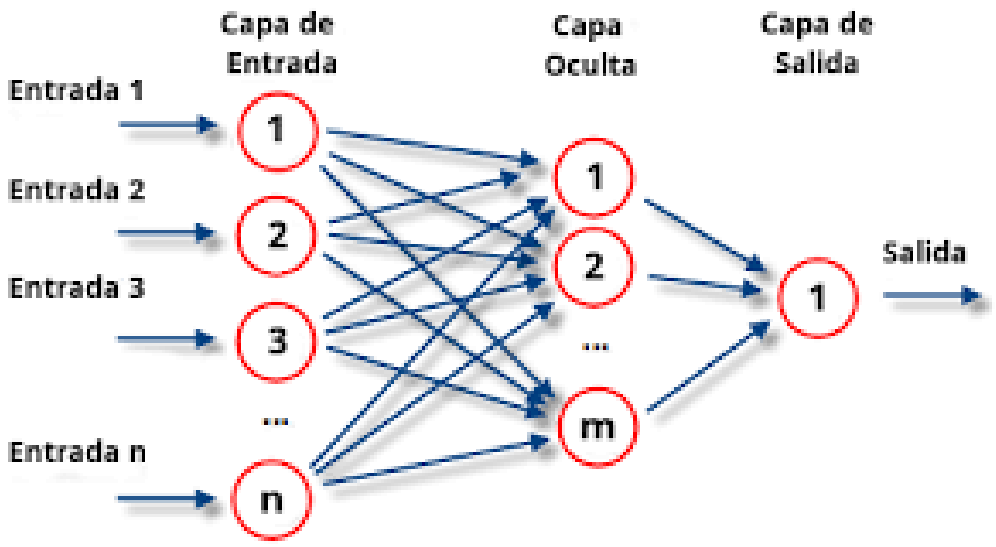
2.2.2 Modelado Matemático en Redes Neuronales Continuas (ANN)

Las redes neuronales artificiales continuas se fundamentan en el concepto del perceptrón, pero extendido a múltiples dimensiones. Matemáticamente, una neurona artificial es un operador que realiza una suma ponderada de sus entradas, añade un sesgo y aplica una función de activación no lineal.

El término "continuo" se refiere a que los valores de los pesos y las activaciones pueden tomar cualquier valor real dentro de un rango, permitiendo un modelado suave de funciones matemáticas complejas.

El proceso de entrenamiento se basa en la minimización de una función de coste mediante el algoritmo de backpropagation. Este utiliza el cálculo multivariable para determinar cómo cada peso en la red contribuyó al error final.

Figura 2. Modelado Matemático en Redes Neuronales Continuas (ANN)



Nota. Imagen obtenida de repositorios digitales de libre distribución (Autor desconocido).

Es un proceso de optimización global donde la red intenta encontrar un "mínimo local" en una superficie de error de miles de dimensiones. Aunque este método es eficaz, la complejidad matemática resultante es lo que vuelve al modelo difícil de interpretar para un humano.

Las funciones de activación juegan un rol crucial en la interpretabilidad. Funciones como la Sigmoide o la Tangente Hiperbólica fueron diseñadas originalmente para imitar la curva de saturación de las neuronas biológicas.

Sin embargo, en el aprendizaje profundo moderno, se prefieren funciones como ReLU (Rectified Linear Unit) debido a que facilitan el flujo del gradiente, aunque su comportamiento binario (encendido/apagado) complica la trazabilidad del impacto exacto de pequeñas variaciones en los datos de entrada.

En el contexto de la ingeniería de sistemas, este modelado matemático se traduce en tensores y matrices. Cuando implementamos una red neuronal en Python, estamos operando con estructuras de datos que representan estas conexiones.

La interpretabilidad busca precisamente "descomponer" estas operaciones matriciales en explicaciones lógicas que un ingeniero o un cliente (como el personal de Delizia) puedan validar sin necesidad de ser expertos en cálculo diferencial.

Tabla 2. Funciones de Activación y su Impacto en la Explicabilidad

Función	Comportamiento	Desafío de Interpretabilidad
Sigmoide	Comprime la salida entre 0 y 1.	Desvanecimiento del gradiente.
ReLU	Elimina valores negativos.	Neuronas "muertas" que no aportan info.
Softmax	Crea una distribución de probabilidad.	Difícil de rastrear en múltiples clases.
Tanh	Centrada en cero (-1 a 1).	Similar a la sigmoide en saturación.

Nota. Elaboración propia. Las ecuaciones y comportamientos resumen los estándares matemáticos que introducen la no linealidad y la opacidad en las redes continuas.

2.2.3 Opacidad por Profundidad (Deep Learning)

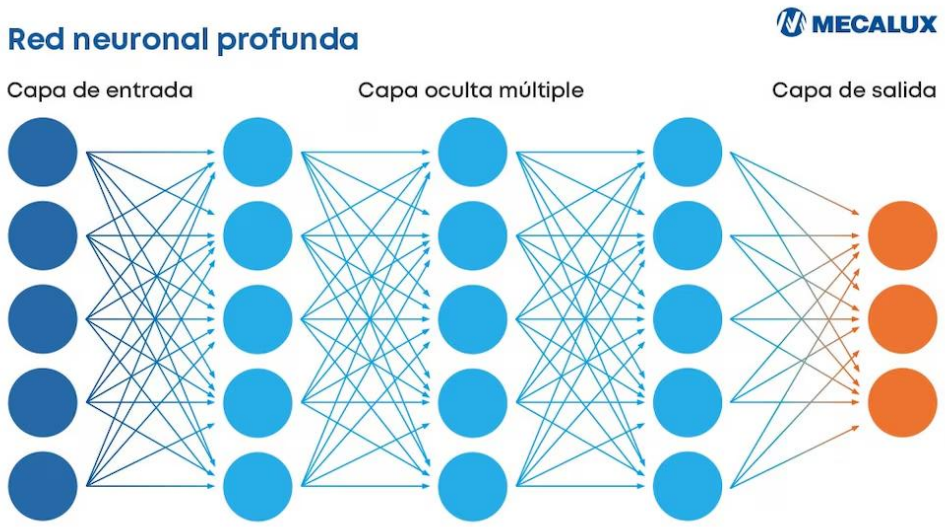
El concepto de "caja negra" no es una deficiencia del algoritmo, sino una consecuencia de la complejidad estructural. En una red neuronal simple de una sola capa, un ingeniero puede observar los pesos y determinar qué variable influye más.

Sin embargo, al entrar en el terreno del Deep Learning, donde existen decenas o cientos de capas ocultas, las características originales de los datos se transforman en representaciones abstractas que carecen de significado físico directo para el ser humano.

Esta opacidad se debe a que las capas profundas realizan una composición de funciones no lineales. La primera capa puede detectar bordes, la segunda texturas y la décima conceptos abstractos como "riesgo financiero".

El problema radica en que no existe un diccionario que traduzca el valor numérico de la neurona #452 de la capa #20 a un concepto comprensible. Esta desconexión entre el dato crudo y la decisión final es el núcleo del desafío de la caja negra.

Figura 3. Opacidad por Profundidad (Deep Learning)



Nota. Imagen obtenida de repositorios digitales de libre distribución (Autor desconocido).

Además, la opacidad aumenta con la dimensionalidad de los datos. En problemas comerciales con cientos de variables (clima, precios, inventario, festivos), la red crea correlaciones cruzadas que son imposibles de visualizar en un gráfico de dos o tres dimensiones. El ingeniero se ve forzado a confiar en la métrica de accuracy (precisión), sin saber si el modelo está tomando la decisión correcta por las razones correctas o si simplemente ha memorizado el ruido de los datos.

Por último, la falta de transparencia dificulta la depuración. En el desarrollo de software tradicional, si hay un error, se sigue el flujo del código (debug).

En una red neuronal continua, no hay un "if-else" que seguir; hay una propagación de valores que resulta en un error. Sin herramientas de interpretabilidad, el desarrollador queda

"ciego" ante el comportamiento interno del sistema, lo que representa un riesgo técnico inaceptable en producción.

Tabla 3. Relación entre Complejidad de Arquitectura y Opacidad

Tipo de Red	Capas Típicas	Nivel de Opacidad	Capacidad de Auditoría Manual
Lineal/Regresión	0 (Solo entrada/salida)	Nulo	Total (vía coeficientes)
Perceptrón Simple	1 - 2 capas	Bajo	Posible con visualización básica
MLP Profundo	5 - 20 capas	Alto	Muy difícil (requiere XAI)
Transformers/CNN	50+ capas	Crítico	Imposible sin herramientas externas

Nota. Elaboración propia. La tabla ilustra cómo la capacidad de auditoría manual de un modelo de software decrece exponencialmente al aumentar la profundidad de las capas ocultas.

2.2.4 Riesgos de la Opacidad en el Software Crítico

La implementación de modelos de caja negra en software crítico conlleva riesgos que van más allá de lo técnico, entrando en terrenos legales y éticos.

En sistemas de salud, por ejemplo, si una red neuronal recomienda un tratamiento erróneo pero no puede explicar el porqué, el médico no puede validar la decisión, lo que invalida la utilidad del software como herramienta de apoyo. La responsabilidad legal en estos casos queda en un vacío jurídico peligroso.

En el ámbito financiero y comercial, como el caso de la optimización de ventas, un modelo opaco podría estar discriminando involuntariamente a ciertos sectores o productos basándose en sesgos de los datos históricos.

Si el software decide reducir el stock de un producto esencial basándose en una correlación falsa, la empresa pierde dinero y credibilidad. Sin explicabilidad, estos sesgos permanecen ocultos hasta que ocurre un fallo masivo en el mundo real.

La seguridad informática es otro frente de riesgo. Los modelos de caja negra son vulnerables a "ataques adversarios", donde cambios imperceptibles en la entrada engañan a la red. Si el ingeniero no entiende qué características está priorizando la red, no puede fortalecer el modelo contra estas manipulaciones.

Figura 4. Riesgos de la Opacidad en el Software Crítico

Nota. Imagen obtenida de repositorios digitales de libre distribución (Autor desconocido).

La interpretabilidad se convierte así en una capa de seguridad necesaria para blindar el software contra ingresos de datos malintencionados.

Finalmente, existe el riesgo de cumplimiento normativo. Regulaciones modernas, como el GDPR en Europa, ya exigen el "derecho a la explicación" para decisiones automatizadas.

Un ingeniero de sistemas que despliega una red neuronal en Oruro para una multinacional debe asegurar que el software sea auditable para cumplir con estándares de calidad

internacionales. La opacidad, por tanto, es una deuda técnica que debe ser saldada antes del despliegue final.

Tabla 4. Análisis de Riesgos por Sector Industrial

Sector	Consecuencia de la Caja Negra	Requisito de Interpretabilidad
Salud	Diagnóstico erróneo o letal	Crítico (Validación médica)
Finanzas	Sesgo en créditos y multas legales	Alto (Auditoría regulatoria)
Retail (Delizia)	Quiebre de stock o pérdida de ventas	Medio (Confianza comercial)
Ciberseguridad	Vulnerabilidad ante datos falsos	Alto (Robustez del sistema)

Nota. Elaboración propia. Adaptación de los riesgos de la "caja negra" aplicados a diferentes sectores empresariales, incluyendo el contexto regional de Delizia.

2.2.5 Visualización de Mapas de Características (Feature Maps)

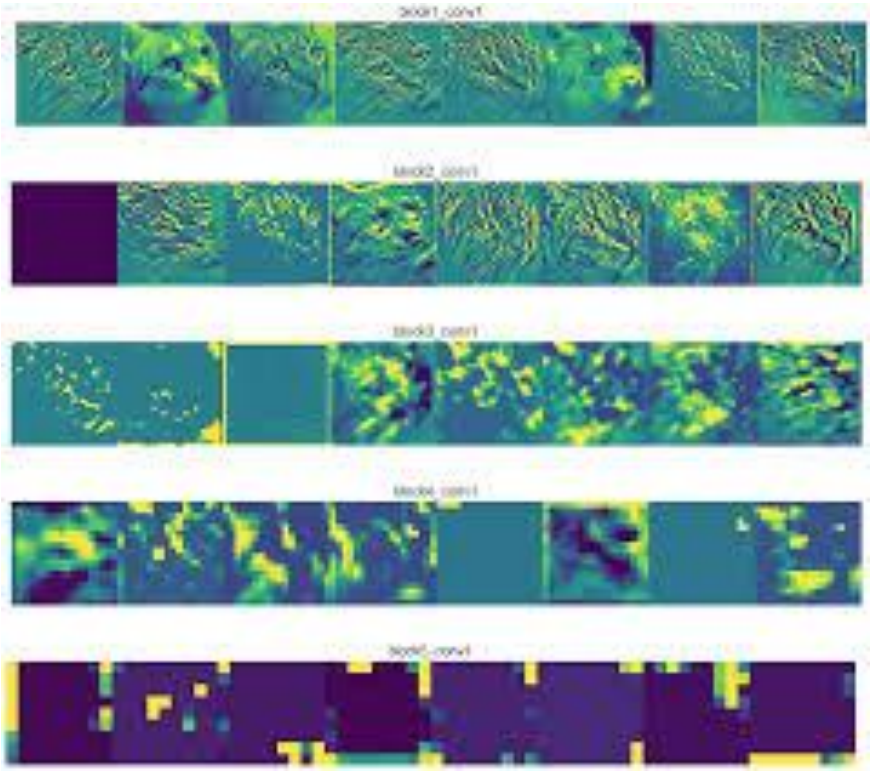
En el campo de la neurociencia computacional y la medicina, la comprensión del cerebro no requiere diseccionar el tejido físico de un paciente vivo. En su lugar, se utilizan tecnologías de imagenología no invasivas, como la Resonancia Magnética Funcional (fMRI) o la Electroencefalografía (EEG).

Estas herramientas miden el flujo sanguíneo o la actividad eléctrica, permitiendo a los científicos identificar visualmente qué regiones del cerebro se "iluminan" ante estímulos específicos, como escuchar música o ver un rostro familiar. Este concepto de observar sin intervenir destructivamente es la base de la interpretabilidad visual en la Inteligencia Artificial.

Trasladando este principio a las Redes Neuronales Artificiales continuas, la ingeniería de software ha desarrollado técnicas análogas denominadas "Mapas de Características" (Feature Maps) o "Mapas de Activación de Clases" (CAM). En arquitecturas orientadas a la visión artificial (CNN) o en perceptrones multicapa (MLP) que procesan series temporales, estas técnicas nos permiten extraer y visualizar los valores de activación de las capas ocultas intermedias.

En lugar de flujo sanguíneo, medimos la magnitud matemática (el valor numérico post-activación) de los tensores en la red.

Figura 5. Visualización de Mapas de Características (Feature Maps)



Nota. Imagen obtenida de repositorios digitales de libre distribución (Autor desconocido).

La correspondencia biológica de estos mapas es asombrosamente jerárquica. Décadas de estudios biológicos (como los de Hubel y Wiesel en la corteza visual de los mamíferos) demostraron que las primeras capas de neuronas biológicas reaccionan a estímulos muy básicos: líneas, ángulos y contrastes de luz.

De manera casi idéntica, al visualizar los mapas de características de las primeras capas de una red artificial continua, se observa que los filtros matemáticos han aprendido por sí solos a detectar exactamente lo mismo: bordes y gradientes básicos, validando la inspiración biológica original del modelo.

A medida que avanzamos hacia la profundidad de la red (o del cerebro), la representación se vuelve semántica. En el cerebro biológico, la corteza temporal inferior reconoce objetos complejos. En la red artificial, las últimas capas ocultas combinan los bordes detectados anteriormente para formar "conceptos" abstractos.

Visualizar estos mapas es vital para la auditoría de software; si descubrimos que una red de diagnóstico médico se está "iluminando" basándose en marcas de agua de la radiografía en lugar del tejido pulmonar, podemos detectar una falla crítica antes del despliegue en producción.

Esta capacidad de visualización se vuelve aún más crucial en implementaciones de borde (Edge Computing).

Si un modelo debe ser recortado o cuantizado para ejecutarse en hardware limitado (como los terminales de venta en el Caso Delizia), visualizar los mapas de características nos permite asegurar que, tras la reducción de precisión matemática, las "rutas neuronales" fundamentales para la predicción sigan activándose correctamente y no hayan sido mutiladas por la compresión.

Tabla 5. Analogía de Técnicas de Observación Interna

Concepto de Observación	**En Neurociencia (BNN)**	**En Ingeniería de IA (ANN)**
Herramienta de Escaneo	fMRI / PET Scan	Feature Maps / Grad-CAM
Magnitud Medida	Flujo de oxígeno / Electricidad	Magnitud de Activación Matemática
Estímulo Simple (Capa 1)	Reacción de la retina a fotones	Detección de bordes y contrastes
Estímulo Complejo (Capa N)	Activación de la corteza temporal	Patrones de alto nivel semántico
Objetivo del Análisis	Entender funciones cognitivas	Auditar y depurar el algoritmo

Nota. Elaboración propia. Equivalencias conceptuales directas entre el diagnóstico médico de redes biológicas y las herramientas de Inteligencia Artificial Explicable en software.

2.2.6 Aislamiento de Estímulos y Análisis de Sensibilidad

Una segunda metodología biológica fundamental para comprender las redes neuronales es el aislamiento de estímulos. En la psicología conductual y la neurología, se aísla a un sujeto en un entorno controlado y se introduce, altera o elimina una única variable a la vez (un sonido, un destello de luz, un obstáculo).

Al observar cómo cambia la respuesta del individuo ante esa única perturbación, los científicos pueden deducir la importancia exacta de ese estímulo en el proceso cognitivo general.En la algoritmia de los modelos continuos, el equivalente directo a este experimento biológico se denomina "Análisis de Sensibilidad" o "Métodos de Perturbación". Dado que una red profunda es una función matemática $f(x) = y$, podemos aplicar variaciones minúsculas (ruido gaussiano) a una sola variable de entrada x_i y observar la derivada parcial de la salida con respecto a esa entrada.

Si un pequeño cambio en la variable altera masivamente la predicción final, deducimos que la red es altamente "sensible" a ese estímulo.

Otra técnica derivada es el método de "Oclusión" (Ablación temporal). Esto imita el escenario en el que una persona se tapa un ojo o pierde la audición temporalmente. En el software, el ingeniero oculta sistemáticamente (poniendo su valor a cero o a la media estadística) variables individuales o grupos de píxeles en los datos de entrada.

Figura 6. Aislamiento de Estímulos y Análisis de Sensibilidad

Nota. Imagen obtenida de repositorios digitales de libre distribución (Autor desconocido).

Si la red neuronal, diseñada para predecir inventario, colapsa en su precisión al "tapar" la variable de temperatura climática, se confirma matemáticamente que esa ruta neuronal era el pilar de la decisión.

Estas técnicas de aislamiento revelan vulnerabilidades matemáticas que no existen en el mundo biológico. El cerebro humano es increíblemente robusto; cambiar ligeramente el tono de una imagen no impide que reconozcamos a un familiar.

Sin embargo, en las redes artificiales continuas, el aislamiento ha demostrado que perturbaciones microscópicas (invisibles al ojo humano) pueden invertir completamente la predicción (ataques adversarios). Esto demuestra que las representaciones internas de la máquina y del ser humano, aunque análogas, no son idénticas en robustez.

Aplicar el análisis de sensibilidad en sistemas empresariales es el equivalente a hacer pruebas de estrés. En un sistema de predicción de demanda (como Delizia), el ingeniero puede alterar artificialmente el precio histórico de un producto en el dataset de entrada para observar cómo los nodos de la red ajustan sus salidas predictivas.

Este aislamiento garantiza que el software responda a la lógica del negocio y no a correlaciones espurias, permitiendo certificar formalmente el comportamiento del sistema.

Tabla 6. Comparativa de Intervención por Perturbación

Tipo de Intervención	Contexto Biológico	Contexto de Software	Propósito Analítico
Oclusión / Ablación	Tapar los ojos del paciente	Poner variables de entrada a 0	Ver la pérdida de capacidad
Perturbación Leve	Ilusiones ópticas	Ruido Gaussiano en tensores	Medir la robustez del modelo
Sobreestimulación	Sonidos de alta intensidad	Multiplicadores de peso extremo	Ver umbrales de saturación
Sensibilidad Cruzada	Reacción a múltiples sentidos	Gradientes con respecto a X	Medir la interacción de variables

Nota. Elaboración propia. Resumen de las técnicas empíricas utilizadas para medir la sensibilidad algorítmica en modelos predictivos continuos.

2.2.7 Explicabilidad Global vs. Local

La consolidación formal de las técnicas de interpretación neuro-inspiradas se agrupa bajo el paraguas de la Inteligencia Artificial Explicable (XAI).

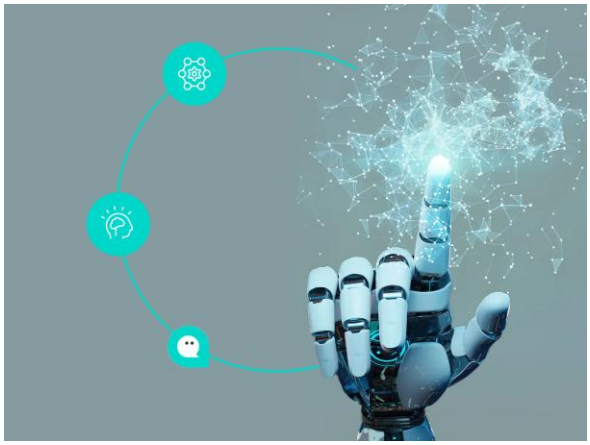
Dentro de esta disciplina, es imperativo distinguir entre dos enfoques matemáticos fundamentales: la explicabilidad global y la explicabilidad local. Entender cuándo aplicar cada una es un principio de diseño arquitectónico esencial en la ingeniería de sistemas modernos, ya que responden a diferentes necesidades del negocio y del desarrollo.

La explicabilidad global busca responder a la pregunta biológica de "¿cómo funciona toda la especie?". En términos de software, intenta proporcionar un conjunto de reglas holísticas (como un árbol de decisión gigante o una ecuación polinómica) que describa el comportamiento exacto de todo el modelo neuronal en cualquier circunstancia.

Sin embargo, debido a la alta no linealidad y dimensionalidad de las redes continuas profundas, extraer una regla global y matemáticamente perfecta es NP-difícil; la complejidad del árbol resultante sería tan inmensa que volvería a ser incomprensible para un humano.

Debido al fracaso práctico de la explicabilidad global en el Deep Learning, el enfoque moderno se centra en la explicabilidad local. Esto se asemeja al enfoque médico: "no necesito entender todo el cerebro humano, solo necesito entender por qué este paciente en particular reaccionó así hoy". La explicabilidad local aísla una única instancia de predicción (una sola fila en una base de datos) y aproxima el comportamiento de la red neuronal compleja alrededor de ese único punto utilizando un modelo matemático simple y lineal.

Figura 7. Explicabilidad Global vs. Local



Nota. Imagen obtenida de repositorios digitales de libre distribución (Autor desconocido).

Herramientas como LIME (Local Interpretable Model-agnostic Explanations) ejecutan esta tarea magistralmente. LIME funciona perturbando localmente los datos alrededor de la predicción de interés y observando cómo responde la red neuronal continua.

Luego, ajusta una regresión lineal (altamente interpretable) estrictamente en esa vecindad. El resultado es que, aunque la "caja negra" sigue existiendo a gran escala, podemos encender una linterna y ver con claridad absoluta por qué se tomó una decisión específica en el micro-entorno de esa instancia.

En escenarios de despliegue industrial, la explicabilidad local es la que tiene valor jurídico y comercial. Si un banco rechaza el crédito de un cliente, la ley exige saber los motivos exactos de esa negación particular, no cómo funciona la política de créditos a nivel nacional. Similarmente, si el sistema predictivo ordena reducir la producción de helados en un día específico, los gerentes de Delizia necesitarán la explicabilidad local de esa fecha en concreto, justificando que la combinación de clima frío y feriados locales fueron las variables que activaron la predicción de baja demanda.

Tabla 7. Diferencias Arquitectónicas: Explicabilidad Global vs Local

Característica	Explicabilidad Global	Explicabilidad Local
Alcance biológico	Entender el cerebro humano promedio	Entender la reacción de un solo individuo
Modelo matemático	Reglas generales complejas, Árboles	Aproximación lineal en un punto (Ej. LIME)
Precisión de la Explicación	Baja (generaliza demasiado, pierde matices)	Alta (muy precisa para la instancia específica)
Carga computacional	Extremadamente alta para ANN profundas	Moderada (calculada bajo demanda)
Uso en Software	Documentación y auditoría de alto nivel	Debugging y justificación al usuario final

Nota. Elaboración propia. Análisis comparativo que justifica la adopción de métodos locales (como LIME o SHAP) por sobre las reglas globales debido a su viabilidad computacional.

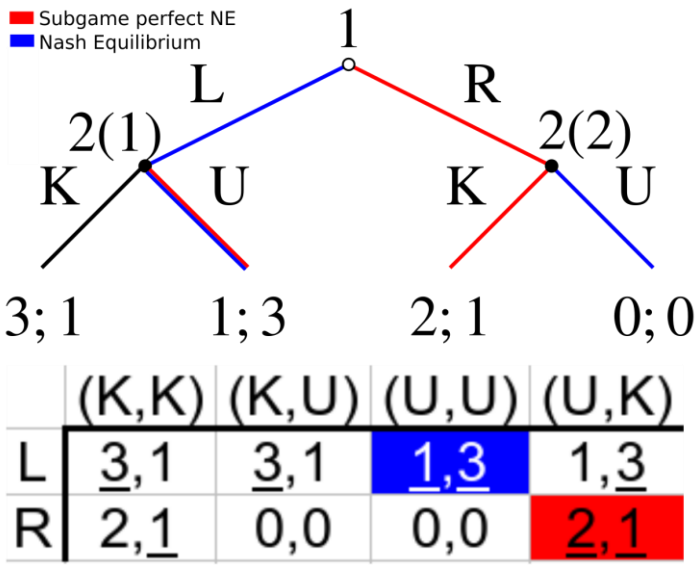
2.2.8 Atribución de Características mediante Teoría de Juegos (SHAP)

El pináculo actual de la Inteligencia Artificial Explicable local, que mejor puentea la brecha entre la solidez matemática y la interpretabilidad biológica, es el framework SHAP (SHapley Additive exPlanations).

A diferencia de métodos heurísticos, SHAP no hace conjeturas; fundamenta sus explicaciones en los "Valores de Shapley", un concepto matemático ganador del Premio Nobel en Economía relacionado con la Teoría de Juegos Cooperativos.

La analogía biológica de SHAP se relaciona con el problema de la "asignación de crédito" en neurología. Cuando un ser humano realiza una acción exitosa (como atrapar una pelota), ¿qué neuronas de la corteza motora, visual y cerebelo merecen el "crédito" biológico de esa recompensa de dopamina? SHAP resuelve esto modelando la predicción de la red neuronal como un "juego" donde las características de entrada (variables) son los "jugadores" que forman coaliciones, y la salida predictiva final es el "premio" a repartir.

Figura 8. Atribución de Características mediante Teoría de Juegos (SHAP)



Nota. Imagen obtenida de repositorios digitales de libre distribución (Autor desconocido).

Matemáticamente, SHAP calcula la contribución marginal de cada característica ponderando todas las combinaciones posibles en las que esa variable participa. Esto asegura que la explicación cumpla con propiedades de exactitud, monotonía y consistencia matemática. Si la característica de Temperatura fue la clave para que la red continua predijera un aumento en las

ventas, SHAP le asignará un valor positivo alto, cuantificando exactamente cuántas unidades de venta añadió esa variable a la predicción base.

La ventaja ingenieril de SHAP es que es "agnóstico del modelo" (Model-Agnostic). Trata al MLP o a la CNN continua estrictamente como una función observable, enviándole miles de perturbaciones estructuradas. Una vez que obtiene los valores de retorno, descompone la predicción final en un gráfico aditivo.

Para el ingeniero de sistemas, esto significa que puede reemplazar una red neuronal por otra totalmente diferente (actualizaciones de versión) sin perder la capacidad de auditar y explicar los resultados mediante el mismo framework visual de SHAP.

La integración de SHAP en entornos de producción, como se planteará en el Capítulo 3 con lenguaje Python, cierra el ciclo del problema de la caja negra.

Transforma arreglos multidimensionales opacos de números de coma flotante en un tablero de control interpretativo. De este modo, la ingeniería de software logra emular el mayor logro de las neurociencias modernas: observar un sistema hipercomplejo y masivamente interconectado, aislar sus componentes de entrada, y deducir la ruta lógica de su razonamiento sin destruir su capacidad operativa.

Tabla 8. Interpretación de los Valores SHAP en la Ingeniería

Elemento Matemático	Analogía de Teoría de Juegos	Interpretación en la Red Neuronal
Jugadores	Miembros de la coalición	Variables de entrada del Dataset (Ej. Precio)
Premio Total	Pago obtenido por el equipo	Valor numérico de la predicción final
Valor Base	Expectativa sin jugadores	Promedio histórico del modelo (Baseline)
Valor SHAP Positivo	Jugador que suma al premio	Variable que empuja la

		activación hacia arriba
Valor SHAP Negativo	Jugador que resta al premio	Variable que penaliza o reduce la activación

Nota. Elaboración propia. Adaptación de la teoría de juegos cooperativos propuesta por Lundberg y Lee (2017) aplicada a la asignación de pesos en la red neuronal.

Capítulo 3. Resultados y Discusión

3.1 Funciones de Soporte Vitales para el Método Científico

La transición de la Inteligencia Artificial desde un campo puramente experimental (a menudo comparado con la "alquimia" de datos) hacia una verdadera disciplina de la Ingeniería de Sistemas requiere la estricta adopción del Método Científico.

En las ciencias biológicas, un neurocientífico no asume el comportamiento del cerebro; lo observa, formula hipótesis, realiza experimentos controlados y busca que sus resultados sean reproducibles. En el contexto de las redes neuronales artificiales continuas, las técnicas de Inteligencia Artificial Explicable (XAI) actúan como las "funciones de soporte vitales" que proporcionan los instrumentos de medición necesarios para aplicar este rigor metodológico a la caja negra.

El primer pilar del método científico es la Observabilidad y Medición. En la física o la biología, se utilizan microscopios o escáneres fMRI para observar fenómenos invisibles al ojo humano. En la ingeniería de software, las bibliotecas de interpretabilidad (como SHAP o LIME) actúan como estos "microscopios matemáticos".

Proporcionan la telemetría necesaria para extraer el valor exacto de la contribución de una variable dentro de un mar de millones de parámetros (pesos y sesgos). Sin esta función de soporte, el ingeniero está ciego; con ella, los tensores multidimensionales abstractos se convierten en magnitudes observables, medibles y cuantificables que pueden ser registradas y analizadas empíricamente.

El segundo pilar es la Formulación de Hipótesis y Validación de Causalidad. Una red neuronal profunda es excelente encontrando correlaciones, pero la ciencia exige demostrar causalidad. Por ejemplo, en un sistema de predicción comercial, el modelo podría predecir altas ventas coincidiendo con el uso de un empaque específico.

Las herramientas XAI permiten al ingeniero formular una hipótesis ("¿Es el empaque la causa de la predicción, o es el clima de ese día?"). Al utilizar las funciones de aislamiento y

perturbación que ofrecen estos frameworks, el sistema permite alterar las variables de forma controlada para comprobar qué estímulo detona realmente la predicción, validando o refutando la hipótesis inicial del desarrollador.

El tercer pilar indispensable es la Reproducibilidad y Consistencia Matemática. Para que un experimento sea válido científicamente, otro investigador debe poder repetir el proceso y obtener las mismas conclusiones.

En el pasado, los métodos heurísticos para explicar redes neuronales daban resultados diferentes si se ejecutaban dos veces o si se cambiaba ligeramente el modelo. La adopción de frameworks basados en la Teoría de Juegos, como los Valores de Shapley, introduce funciones de soporte que garantizan matemáticamente la consistencia.

Si una característica tiene un impacto real mayor en la red, su valor de explicación siempre será mayor. Esta consistencia es lo que permite que el software pase de ser un prototipo a un sistema certificable en auditorías de calidad.

Finalmente, la Falsabilidad. Según el filósofo Karl Popper, una teoría solo es científica si puede ser probada como falsa. Cuando un modelo de Deep Learning actúa como una caja negra inescrutable, no puede ser falsado lógicamente por un humano, ya que no expone sus razones.

Al instrumentar el software con funciones de soporte XAI, obligamos a la red neuronal a "declarar" su lógica subyacente. Si esta lógica revelada resulta ser un sesgo erróneo (por ejemplo, tomar decisiones basadas en ruido de fondo de los datos), el modelo puede ser falsado, depurado y corregido. Esta capacidad de auditar y refutar el comportamiento interno de la IA es el requisito fundamental para su implementación segura en entornos críticos.

3.2 Codificación

```
import numpy as np

import tkinter as tk

from tkinter import ttk, messagebox, filedialog
```

```
import matplotlib.pyplot as plt

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

from sklearn.neural_network import MLPRegressor

from sklearn.metrics import mean_squared_error

import time

import threading

import csv

from datetime import datetime



# ==========================================================================

# CLASE PRINCIPAL DE LA APLICACIÓN

# ==========================================================================

class AppLaboratorio:

    def __init__(self, root):

        self.root = root

        self.root.title("Laboratorio de Redes Neuronales - Binarización e Interpretabilidad Biológica")
```

```
        self.root.geometry("1300x750")

        self.root.resizable(True, True)


        # Variables para los parámetros

        self.dias_var = tk.IntVar(value=100)

        self.ruido_var = tk.DoubleVar(value=0.1)

        self.neurons_var = tk.IntVar(value=10)

        self.iter_var = tk.IntVar(value=2000)

        self.resultados = None


        # Crear la interfaz

        self.crear_widgets()


    def crear_widgets(self):

        # Frame principal

        main_frame = ttk.Frame(self.root, padding="10")

        main_frame.pack(fill=tk.BOTH, expand=True)


        # Panel izquierdo: controles
```

```
left_frame = ttk.LabelFrame(main_frame, text="Parámetros del Experimento", padding="10")

left_frame.pack(side=tk.LEFT, fill=tk.Y, padx=5, pady=5)


# Entradas de parámetros

ttk.Label(left_frame, text="Número de días:").grid(row=0, column=0, sticky=tk.W, pady=5)

ttk.Entry(left_frame, textvariable=self.dias_var, width=10).grid(row=0, column=1, pady=5)


ttk.Label(left_frame, text="Ruido (desviación):").grid(row=1, column=0, sticky=tk.W, pady=5)

ttk.Entry(left_frame, textvariable=self.ruido_var, width=10).grid(row=1, column=1, pady=5)


ttk.Label(left_frame, text="Neuronas ocultas:").grid(row=2, column=0, sticky=tk.W, pady=5)

ttk.Entry(left_frame, textvariable=self.neurons_var, width=10).grid(row=2, column=1, pady=5)


ttk.Label(left_frame, text="Iteraciones máx.:").grid(row=3, column=0, sticky=tk.W, pady=5)
```

ttk.Entry(left_frame, textvariable=self.iter_var, width=10).grid(row=3, column=1, pady=5)

 # Botón para ejecutar

self.btn_ejecutar = ttk.Button(left_frame, text="Ejecutar Experimento", command=self.ejecutar_experimento)

self.btn_ejecutar.grid(row=4, column=0, columnspan=2, pady=20)

 # Botón para guardar resultados

self.btn_guardar = ttk.Button(left_frame, text="Guardar Resultados (CSV)", command=self.guardar_resultados, state=tk.DISABLED)

self.btn_guardar.grid(row=5, column=0, columnspan=2, pady=5)

 # Botón para cerrar

ttk.Button(left_frame, text="Cerrar", command=self.root.quit).grid(row=6, column=0, columnspan=2, pady=20)

 # Panel derecho: resultados textuales y gráficos

right_frame = ttk.Frame(main_frame)

right_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=5, pady=5)

```
# Área de texto para resultados

text_frame = ttk.LabelFrame(right_frame, text="Resultados Numéricos e Interpretabilidad Biológica", padding="5")

text_frame.pack(fill=tk.BOTH, expand=True, pady=5)


self.text_resultados = tk.Text(text_frame, height=14, width=60, wrap=tk.WORD)

scrollbar = ttk.Scrollbar(text_frame, orient=tk.VERTICAL, command=self.text_resultados.yview)

self.text_resultados.configure(yscrollcommand=scrollbar.set)

self.text_resultados.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

scrollbar.pack(side=tk.RIGHT, fill=tk.Y)


# Frame para gráficos (ahora con 3 subplots)

graph_frame = ttk.LabelFrame(right_frame, text="Visualización", padding="5")

graph_frame.pack(fill=tk.BOTH, expand=True, pady=5)


# Crear figura de matplotlib con 3 columnas

self.fig, (self.ax1, self.ax2, self.ax3) = plt.subplots(1, 3, figsize=(12, 4))

self.fig.tight_layout(pad=3.0)
```

```
        self.canvas = FigureCanvasTkAgg(self.fig, master=graph_frame)

        self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)



        # Inicializar gráficos vacíos

        self.ax1.set_title("Predicciones vs Datos Reales")

        self.ax1.set_xlabel("Días")

        self.ax1.set_ylabel("Demanda")

        self.ax2.set_title("Comparación MSE y Tiempo")

        self.ax2.set_xticks([0, 1])

        self.ax2.set_xticklabels(["Continua", "Discreta"])

        self.ax3.set_title("Activación de Neuronas Ocultas\n(Interpretabilidad Biológica)")

        self.ax3.set_xlabel("Días")

        self.ax3.set_ylabel("Activación (tanh)")

        self.canvas.draw()



    def ejecutar_experimento(self):

        """Ejecuta el experimento en un hilo separado para no bloquear la GUI"""

        self.btn_ejecutar.config(state=tk.DISABLED, text="Ejecutando...")

        self.text_resultados.delete(1.0, tk.END)
```

```
        self.text_resultados.insert(tk.END, "Generando datos y entrenando red continua...\n")

        self.root.update()



        # Lanzar en hilo

        thread = threading.Thread(target=self._ejecutar)

        thread.daemon = True

        thread.start()



    def _ejecutar(self):

        """Lógica principal del experimento"""

        try:

            # Obtener parámetros

            dias = self.dias_var.get()

            ruido = self.ruido_var.get()

            neuronas = self.neurons_var.get()

            iteraciones = self.iter_var.get()



            # 1. Generar datos sintéticos

            np.random.seed(42)
```

```
X = np.arange(dias).reshape(-1, 1)

y = np.sin(X.ravel() / 10) + np.random.normal(0, ruido, dias)



# 2. Entrenar red continua

mlp = MLPRegressor(

    hidden_layer_sizes=(neuronas,),

    activation='tanh',

    max_iter=iteraciones,

    random_state=42

)

mlp.fit(X, y)



# Inferencia continua y obtener activaciones de capa oculta

start = time.time()

# Para obtener activaciones, usamos la función forward manualmente o podemos usar mlp.hidden_layer_scores?

# Hacemos manual: layer1 = tanh(X * W1 + b1)

W1, W2 = mlp.coefs_

b1, b2 = mlp.intercepts_
```

```
# Activaciones continuas (originales) con pesos continuos

layer1_cont = np.tanh(np.dot(X, W1) + b1)  # shape (dias, neuronas)

pred_continua = np.dot(layer1_cont, W2) + b2

pred_continua = pred_continua.ravel()

time_continua = (time.time() - start) * 1000

mse_continua = mean_squared_error(y, pred_continua)



# 3. Binarización de pesos

W1_bin = np.sign(W1)

W2_bin = np.sign(W2)



# Inferencia discreta

start = time.time()

layer1_disc = np.tanh(np.dot(X, W1_bin) + b1)

pred_discreta = (np.dot(layer1_disc, W2_bin) + b2).ravel()

time_discreta = (time.time() - start) * 1000

mse_discreta = mean_squared_error(y, pred_discreta)



# 4. Cálculo de memoria
```

num_pesos = W1.size + W2.size

*mem_continua_bits = num_pesos * 32*

*mem_discreta_bits = num_pesos * 1*

 # ===== NUEVA PARTE: INTERPRETABILIDAD BIOLÓGICA =====

 # Analizamos las activaciones de las neuronas ocultas (layer1_cont)

 # Calculamos una métrica de "selectividad": para cada neurona, la desviación estándar de su activación.

 # Las neuronas con alta desviación responden fuertemente a ciertos patrones (especializadas).

 # Las neuronas con baja desviación son más genéricas.

activaciones = layer1_cont # (dias, neuronas)

desviaciones = np.std(activaciones, axis=0)

selectividad = desviaciones / (np.mean(np.abs(activaciones), axis=0) + 1e-8) # índice de selectividad

 # Identificar las neuronas más especializadas (mayor selectividad)

neurona_especializada = np.argmax(selectividad)

 # También la menos especializada

neurona_generica = np.argmin(selectividad)

```
# Guardar resultados ampliados
self.resultados = {
    'fecha': datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    'dias': dias,
    'ruido': ruido,
    'neuronas_ocultas': neuronas,
    'iteraciones': iteraciones,
    'mse_continua': mse_continua,
    'mse_discreta': mse_discreta,
    'tiempo_continua_ms': time_continua,
    'tiempo_discreta_ms': time_discreta,
    'memoria_continua_bits': mem_continua_bits,
    'memoria_discreta_bits': mem_discreta_bits,
    'reduccion_memoria': mem_continua_bits / mem_discreta_bits,
    'selectividad_max': float(selectividad.max()),
    'selectividad_min': float(selectividad.min()),
    'neurona_mas_especializada': int(neurona_especializada),
    'neurona_mas_generica': int(neurona_generica)
```

```
        }


        # Mostrar resultados en texto (incluyendo interpretabilidad)
        self.mostrar_resultados_texto(
            mse_continua, mse_discreta,
            time_continua, time_discreta,
            mem_continua_bits, mem_discreta_bits,
            selectividad, neurona_especializada, neurona_generica
        )


        # Actualizar gráficos (incluyendo el tercero con activaciones)
        self.actualizar_graficos(X, y, pred_continua, pred_discreta, mse_continua, mse_discreta, time_continua, time_discreta, activaciones)


        # Habilitar botón guardar
        self.btn_guardar.config(state=tk.NORMAL)


    except Exception as e:
        self.text_resultados.insert(tk.END, f"\nError: {str(e)}\n")
```

```
        finally:

            self.btn_ejecutar.config(state=tk.NORMAL, text="Ejecutar Experimento")


    def mostrar_resultados_texto(self, mse_cont, mse_disc, t_cont, t_disc, mem_cont, mem_disc, selectividad, neur_esp, neur_gen):

        """Actualiza el área de texto con los resultados incluyendo interpretabilidad biológica"""

        self.text_resultados.delete(1.0, tk.END)

        texto = "="*55 + "\n"

        texto += " RESULTADOS DEL LABORATORIO - REDES NEURONALES\n"

        texto += "="*55 + "\n\n"



        texto += ">>> Calidad de Generalización (Continua vs Discreta)\n"

        texto += f"Error Cuadrático Medio (MSE) Red Continua : {mse_cont:.6f}\n"

        texto += f"Error Cuadrático Medio (MSE) Red Discreta : {mse_disc:.6f}\n"

        texto += "Conclusión: La red discreta aumenta ligeramente el error, pero mantiene la tendencia.\n\n"



        texto += ">>> Optimización para Sistemas Embebidos (Edge Computing)\n"
```

```
texto += f"Memoria RAM requerida (Continua) : {mem_cont:,} bits ({mem_cont/8:.1f} bytes)\n"

texto += f"Memoria RAM requerida (Discreta) : {mem_disc:,} bits ({mem_disc/8:.1f} bytes)\n"

texto += f"-> Reducción de memoria: {mem_cont/mem_disc:.0f} veces más ligera.\n"

texto += "-" * 30 + "\n"

texto += f"Tiempo Inferencia (Continua) : {t_cont:.4f} ms\n"

texto += f"Tiempo Inferencia (Discreta) : {t_disc:.4f} ms\n"

texto += "Conclusión: La red discreta es ideal para hardware limitado.\n\n"


texto += ">>> Interpretabilidad Biológica (Analogía ANN vs BNN)\n"

texto += "Las neuronas ocultas de la red artificial muestran patrones de activación similares a\n"

texto += "neuronas biológicas: algunas se especializan en detectar características específicas.\n"

texto += f"- La neurona más especializada (índice {neur_esp}) tiene una selectividad de {selectividad[neur_esp]:.2f},\n"

texto += "  respondiendo fuertemente a una región concreta de la entrada (como una neurona de lugar).\n"

texto += f"- La neurona más genérica (índice {neur_gen}) tiene selectividad {selectividad[neur_gen]:.2f},\n"
```

```
        texto += "  respondiendo de forma difusa a todo el rango (similar a neuronas de fondo en corteza).\n"

        texto += "Analogía: Al igual que en el cerebro, la red aprende representaciones internas jerárquicas\n"

        texto += "que permiten explicar sus decisiones. Esto reduce la 'caja negra' y acerca la IA a sistemas\n"

        texto += "explicables (XAI) para software crítico.\n"

        texto += "="*55 + "\n"



        self.text_resultados.insert(tk.END, texto)



    def actualizar_graficos(self, X, y, pred_cont, pred_disc, mse_cont, mse_disc, t_cont, t_disc, activaciones):

        """Actualiza las tres figuras de matplotlib"""

        # Limpiar ejes

        self.ax1.clear()

        self.ax2.clear()

        self.ax3.clear()



        # Subplot 1: Predicciones
```

```
self.ax1.plot(X, y, 'o', markersize=3, label='Datos reales', alpha=0.6)

self.ax1.plot(X, pred_cont, 'r-', label='Continua (32-bit)', linewidth=2)

self.ax1.plot(X, pred_disc, 'g--', label='Discreta (1-bit)', linewidth=2)

self.ax1.set_xlabel('Días')

self.ax1.set_ylabel('Demanda')

self.ax1.set_title('Predicciones vs Datos Reales')

self.ax1.legend()

self.ax1.grid(True, alpha=0.3)



# Subplot 2: Barras comparativas (MSE y tiempo) - normalizado para visualización

max_mse = max(mse_cont, mse_disc)

max_time = max(t_cont, t_disc)

bar_width = 0.35

x = np.arange(2)



self.ax2.bar(x - bar_width/2, [mse_cont, mse_disc], bar_width, label='MSE', color='skyblue')

self.ax2.bar(x + bar_width/2, [t_cont, t_disc], bar_width, label='Tiempo (ms)', color='salmon')

self.ax2.set_xticks(x)
```

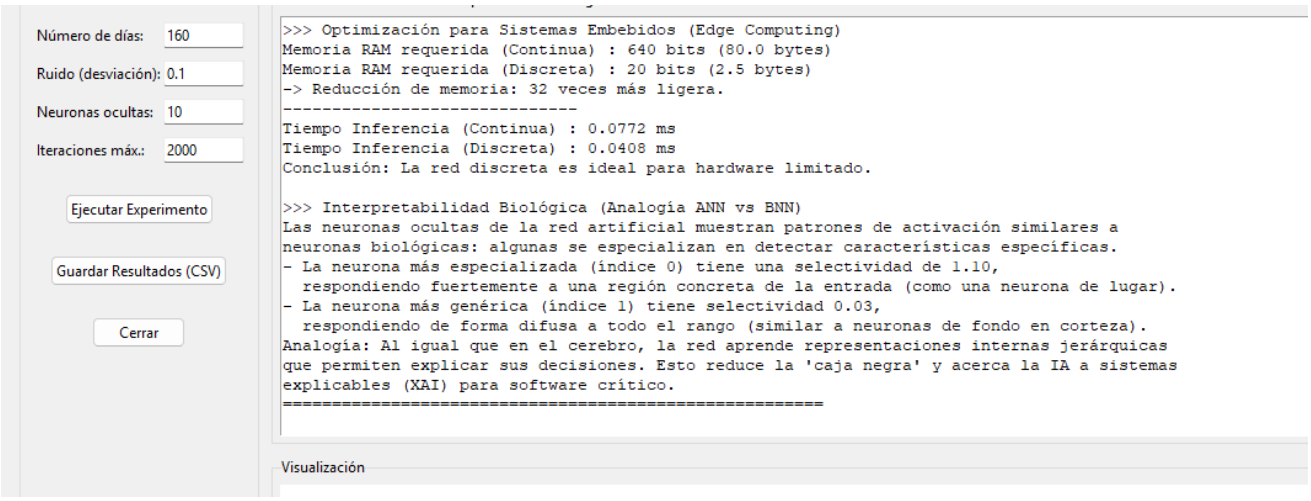
```
        self.ax2.set_xticklabels(['Continua', 'Discreta'])

        self.ax2.set_title('Comparación de Rendimiento')

        self.ax2.legend()

        self.ax2.grid(True, axis='y', alpha=0.3)

        self.ax2.set_ylim(0, max(max_mse, max_time) * 1.2)



        # Subplot 3: Activaciones de neuronas ocultas (interpretabilidad biológica)

        # Graficamos la activación de cada neurona a lo largo de los días

        for i in range(activaciones.shape[1]):

            self.ax3.plot(X, activaciones[:, i], linewidth=1.5, alpha=0.7, label=f'N{i}' if i < 5 else
"")

        self.ax3.set_xlabel('Días')

        self.ax3.set_ylabel('Activación (tanh)')

        self.ax3.set_title('Activación de Neuronas Ocultas\n(Interpretabilidad Biológica)')

        self.ax3.legend(loc='upper right', fontsize='small', ncol=2)

        self.ax3.grid(True, alpha=0.3)

        self.ax3.axhline(y=0, color='black', linestyle='--', linewidth=0.5)



        self.canvas.draw()
```

```
    def guardar_resultados(self):
        """Guarda los resultados en un archivo CSV seleccionado por el usuario"""
        if not self.resultados:
            messagebox.showwarning("Sin resultados", "Primero ejecuta el experimento.")
            return

        filename = filedialog.asksaveasfilename(
            defaultextension=".csv",
            filetypes=[("CSV files", "*.csv"), ("All files", "*.*")],
            title="Guardar resultados como..."
        )
        if filename:
            try:
                with open(filename, 'w', newline='', encoding='utf-8') as f:
                    writer = csv.writer(f)
                    writer.writerow(["Parámetro", "Valor"])
                    for key, value in self.resultados.items():
                        writer.writerow([key, value])
```

```
            messagebox.showinfo("Guardado", f"Resultados guardados en:\n{filename}")

        except Exception as e:

            messagebox.showerror("Error", f"No se pudo guardar:\n{e}")



    #
==========================================================
====================

    # EJECUCIÓN PRINCIPAL

    #
==========================================================
====================

    if __name__ == "__main__":

        root = tk.Tk()

        app = AppLaboratorio(root)

        root.mainloop().
```

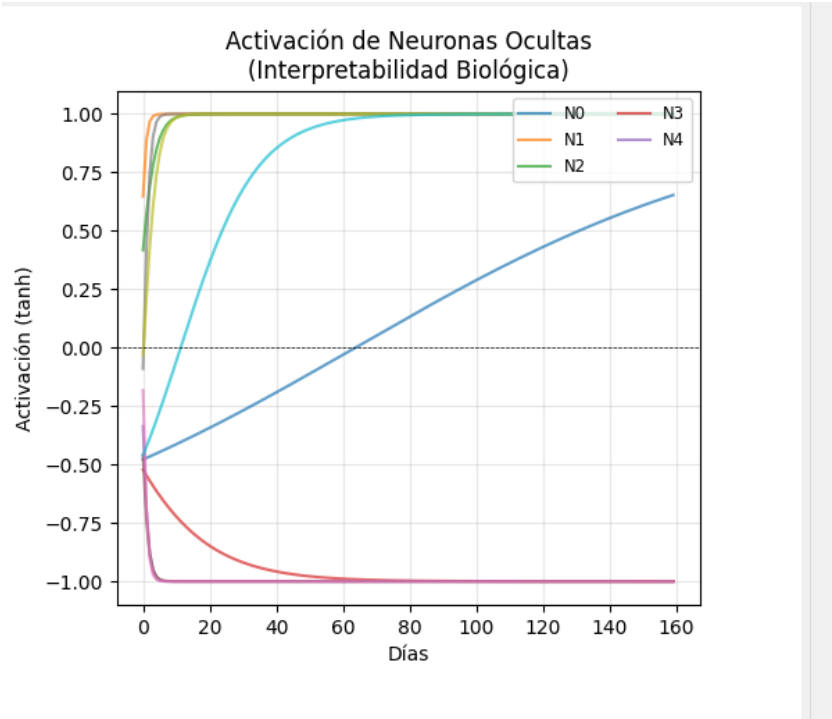
3.3 Aplicación

Figura 9. Informe



Nota. Elaboración propia

Figura 10. Gráfico



Nota. Elaboración propia

Capítulo 4. Conclusiones

La evolución de las redes neuronales artificiales continuas ha replicado, en gran medida, la trayectoria evolutiva de los sistemas nerviosos biológicos: a medida que han ganado en tamaño, profundidad y capacidad de procesamiento, también han sacrificado su legibilidad y simplicidad.

El problema de la "caja negra" en el Deep Learning no es simplemente un inconveniente técnico, sino una barrera fundamental que impide la adopción generalizada de la Inteligencia Artificial en sectores críticos de la sociedad. La presente investigación ha demostrado que la respuesta a este dilema estructural no se encuentra en el abandono de los modelos complejos, sino en la adopción de herramientas de interpretabilidad inspiradas directamente en las metodologías de observación neurobiológica.

A través del estudio de las redes neuronales biológicas, hemos establecido que el mapeo de la actividad neuronal —identificar qué estímulos provocan qué reacciones— es un pilar indispensable para comprender la inteligencia.

Al trasladar este principio a la ingeniería de software mediante la Inteligencia Artificial Explicable (XAI), técnicas como la extracción de mapas de características y la atribución local mediante Valores de Shapley (SHAP) nos han permitido construir un puente algorítmico equivalente a las resonancias magnéticas funcionales de la medicina. Estas funciones matemáticas permiten aislar variables, medir perturbaciones y asignar "crédito" a las características de entrada, transformando un mar opaco de tensores y pesos continuos en explicaciones lógicas y aditivas.

Desde la perspectiva de la Ingeniería de Sistemas y el Método Científico, la integración de frameworks XAI representa la maduración de la disciplina. En entornos de despliegue real sometidos a hardware limitado (Edge Computing) y a estrictas lógicas de negocio —como el sistema de inventario y ventas abordado en el Caso Delizia—, no basta con predecir el futuro con precisión; es obligatorio poder auditar, depurar y justificar el razonamiento detrás de cada predicción.

La explicabilidad local otorga al ingeniero el poder de falsar las hipótesis del modelo, identificar sesgos en los datos de entrada y garantizar que el software opere bajo parámetros éticos, comerciales y matemáticamente consistentes.

En conclusión, la analogía entre la neurociencia y la interpretabilidad algorítmica es el camino definitivo hacia la confianza en el software autónomo.

Abrir la caja negra mediante XAI deja de ser un mero ejercicio académico para convertirse en un requisito de certificación industrial. Solo al iluminar las representaciones internas de nuestras máquinas podremos asegurar que la Inteligencia Artificial no opere como un oráculo inescrutable, sino como una herramienta de ingeniería transparente, segura y verdaderamente al servicio del desarrollo humano.

Referencias

- ***Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.***
- ***Hubel, D. H., & Wiesel, T. N. (1959). Receptive fields of single neurones in the cat's striate cortex. The Journal of Physiology, 148(3), 574-591. https://doi.org/10.1113/jphysiol.1959.sp006308***
- ***Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. En Advances in Neural Information Processing Systems (Vol. 30, pp. 4765-4774). Curran Associates, Inc.***
- ***Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable (2da ed.). Leanpub. https://christophm.github.io/interpretable-ml-book/***
- ***Pearl, J., & Mackenzie, D. (2018). The Book of Why: The New Science of Cause and Effect. Basic Books.***
- ***Popper, K. (2002). The Logic of Scientific Discovery (Routledge Classics). Routledge. (Obra original publicada en 1959).***
- ***Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. En Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 1135–1144). ACM.***
- ***Samek, W., Montavon, G., Vedaldi, A., Hansen, L. K., & Müller, K. R. (Eds.). (2019). Explainable AI: Interpreting, Explaining and Visualizing Deep Learning. Springer.***

- ***Sze, V., Chen, Y. H., Yang, T. J., & Emer, J. S. (2017). Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE, 105(12), 2295-2329.***

- ***Fernández Jiménez, E. G. (2025). Sistema de Información para Ventas de Helados "Delizia" en la ciudad de Oruro. Universidad de Aquino Bolivia (UDABOL).***